INFR11212: Computer Vision Project

1 Introduction

This report presents work on two complementary tasks using the PUMA dataset [1]: tissue segmentation at the slide level and nuclei classification at the cell level. These tasks operate at different scales and require different modelling strategies, but both demand careful handling of class imbalance, limited labelled data, and morphological variability inherent to H&E stained tissue.

For Task 1, we implement and compare an end-to-end UNet segmentation model against an autoencoder pre-training approach. For Task 2, we implement a supervised ResNet18 classifier and compare it against a supervised contrastive learning approach. In both tasks, we evaluate the hypothesis that representation learning through unsupervised or self-supervised pre-training improves downstream performance on small labelled datasets.

2 Task 1: Image Segmentation

2.1 Dataset Preprocessing

The PUMA dataset [1] provides H&E stained ROIs in `.tif` format with polygon annotations as `.GeoJSON` files, pre split into 163 training, 20 validation, and 22 test images. Following the task specification, six raw tissue classes were consolidated into three: Tissue Tumour (label 1), Tissue Stroma (label 2), and Other (label 0), where Other subsumes Blood Vessel, Epidermis, White Background, and Necrosis. This grouping is necessary because each minority type alone is too rare to support reliable learning.

Polygon annotations were converted to pixel masks using `rasterio` with an identity affine transform matching QuPath's [2] pixel space coordinates. Overlapping regions were resolved by painting in ascending clinical priority: Other first, then Stroma, then Tumour last, since tumour nests are physically embedded within stroma and must take label precedence. Masks were cached as `.npy` files.

A pixel level audit revealed severe imbalance (Table 1). Tumour dominates at 69.1%, while Other covers only 5.5% and merges four visually distinct tissue types, making it structurally the hardest class.

Table 1: Pixel distribution across 163 training images with computed CE weights (raw inverse frequency, capped at 6×).

Class	Pixel Count	%	CE Weight
Other (0)	9,458,492	5.5%	2.309
Tumour (1)	118,044,882	69.1%	0.186
Stroma (2)	43,414,514	25.4%	0.505

Training used random 512×512 crops from the original images, preserving full resolution tissue detail. Each image yielded 12 patches per epoch (1,956 total), with a class aware sampler centring patches on specific classes with weights 40% Other, 30% Tumour, 30% Stroma. This sampling strategy directly counteracts spatial imbalance by ensuring the model encounters rare tissue types more frequently.

The pipeline used `albumentations` [3] and included: horizontal and vertical flips (p=0.5), random 90 rotations (p=0.5), elastic transforms (α=60, σ=6, p=0.3), grid distortion (p=0.3), and colour jitter (brightness/contrast ±0.15, p=0.5). Colour jitter was particularly important for H&E data since staining intensity varies between laboratories [4]. All transforms were applied identically to both image and mask. Crucially, all images were kept in $[0, 1]$ range throughout training and inference with no ImageNet normalisation, ensuring the encoder saw the same input distribution in both the autoencoder and segmentation stages.

3 Network Architectures

3.1 UNet: End to End Segmentation

A five level UNet [5] was implemented from scratch (Figure 1). The encoder uses four `DoubleConv` blocks, each comprising two Conv→BatchNorm [6]→ReLU sequences separated by 2×2 max pooling. Two consecutive 3×3 convolutions achieve the same receptive field as a single 5×5 with fewer parameters (2×9=18 vs 25 per channel pair). Channel counts follow $48\rightarrow96\rightarrow192\rightarrow384$ with a 768 channel bottleneck at $1/16$th spatial resolution. The decoder uses transposed convolutions for learned upsampling, with skip connections concatenating encoder features at each level to restore spatial detail lost during downsampling. A final 1×1 convolution produces three class logits. Starting

at 48 base channels yields 17,461,491 trainable parameters, roughly 7× fewer than the ∼125M baseline, a deliberate capacity choice since a larger model would overfit severely on 163 images [7].

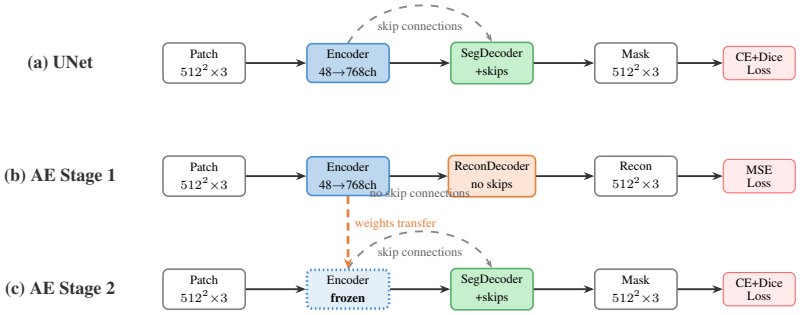


Figure 1: Overview of all pipelines. (a) UNet trains encoder and decoder end to end with skip connections. (b) AE Stage 1 trains the encoder via reconstruction without skips, forcing a compressed bottleneck. (c) AE Stage 2 freezes the pre trained encoder (dotted border) and trains only the SegDecoder with skips restored. Both DoubleConv blocks use Conv→BN→ReLU×2; channels progress 48→96→192→384→768.

3.2 Autoencoder Pre training for Segmentation

The second approach follows a two stage paradigm [8]. In **Stage 1**, an encoder identical to the UNet encoder is paired with a `ReconDecoder` and trained on 512×512 patches without labels, minimising MSE between input and reconstruction. The `ReconDecoder` deliberately omits skip connections, forcing the 768 channel bottleneck to encode a genuinely compressed representation rather than bypassing compression. In **Stage 2**, the pre trained encoder is *frozen* and a fresh `SegDecoder` (with skip connections restored) is trained on labelled masks. Freezing the encoder prevents catastrophic forgetting on the small labelled dataset. The hypothesis is that reconstruction features accelerate convergence and improve accuracy. Parameter counts are reported in Table 2.

Table 2: Trainable parameter counts. The frozen encoder in Stage 2 contributes no gradient updates.

Model	Trainable Params	vs Baseline
Baseline	∼125,000,000	reference
UNet (end to end)	17,461,491	7× fewer
Autoencoder (Stage 1)	15,698,931	8× fewer
SegDecoder only (Stage 2)	6,858,147	18× fewer
Encoder (frozen, Stage 2)	10,603,344	not trained

4 Training Methodology

4.1 Loss Function

Both models were trained with a combined loss:

$$\mathcal{L} = 0.5 \cdot \mathcal{L}_{\text{CE}} + 0.5 \cdot \mathcal{L}_{\text{Dice}} \tag{1}$$

The weighted cross entropy penalises each pixel independently: $\mathcal{L}_{\text{CE}} = -\sum_i \sum_c w_c \cdot y_{i,c} \cdot \log \hat{p}_{i,c}$, where w_c are the class weights from Table 1. However, cross entropy alone is insufficient for imbalanced segmentation because a model can achieve low loss by predicting Tumour everywhere (69.1% of pixels). The soft Dice loss addresses this by measuring region level overlap [9]:

$$\mathcal{L}_{\text{Dice}} = 1 - \frac{1}{C} \sum_{c=0}^{C-1} \frac{2 \sum_i p_{i,c} \cdot g_{i,c} + \epsilon}{\sum_i p_{i,c} + \sum_i g_{i,c} + \epsilon} \tag{2}$$

If the model predicts zero Other pixels, Dice loss returns 1.0 for that class, providing corrective gradient signal that cross entropy cannot deliver. Class weights were computed from the raw cached masks (not from the patch loader, which is already biased by the stratified sampler, and would cause double correction). Raw inverse frequency weights were capped at 6× to prevent gradient instability.

4.2 Optimiser and Scheduling

Both models used Adam [10] with learning rate 10^{-3}, weight decay 10^{-4}, gradient clipping at 1.0, and automatic mixed precision (AMP). A `ReduceLROnPlateau` scheduler halved the learning rate after 4 epochs without improvement, with minimum 10^{-6}. An EMA smoothing (α=0.25) was applied to the validation Dice signal before feeding it to the scheduler, absorbing the high epoch to epoch variance caused by the 20 image validation set. Early stopping patience was set to 30 epochs. The UNet trained for 120 epochs; Stage 1 ran for 50 epochs and Stage 2 for up to 100 epochs.

4.3 Inference

All validation and test evaluation used tiled inference: each image was processed through overlapping 512×512 tiles with stride 384, and logits were averaged across overlapping regions before argmax. This preserves native resolution and avoids aliasing artefacts from resizing. Global Dice and IoU were computed by accumulating intersections and unions across all images before computing the final ratio, which is more robust than averaging per image scores.

5 Results

5.1 Training Dynamics

Figure 2 shows the UNet training curves. The validation Dice steadily improved over 120 epochs, surpassing the baseline around epoch 40 and reaching a best of 0.5026 at epoch 102. Notable is the high epoch to epoch variance in validation metrics (swings of up to 0.10), which directly reflects the instability of evaluating on only 20 images. The EMA smoothing (dashed green) effectively filters this noise for the scheduler. The per class panel reveals that Tumour Dice stabilises around 0.80 early on, while Other remains volatile and low, reflecting the difficulty of learning a semantically incoherent class.

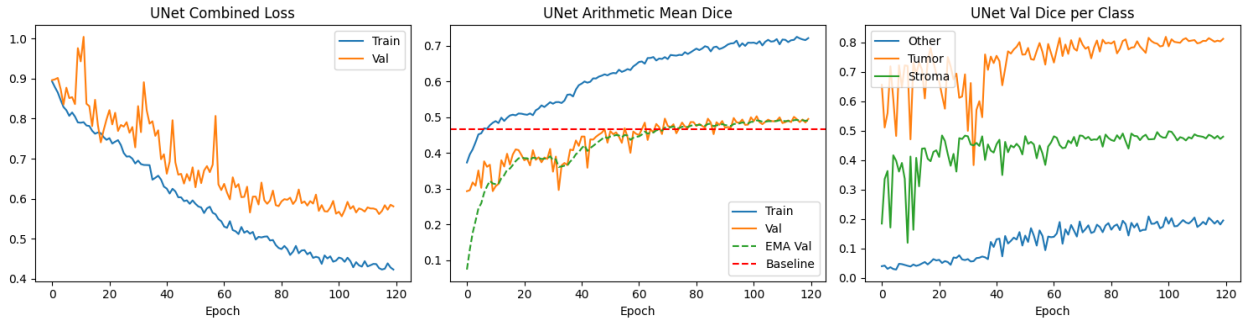


Figure 2: UNet training curves. Left: combined loss. Centre: mean Dice with baseline (dashed red) and EMA smoothed signal (dashed green). Right: per class validation Dice showing Tumour consistently highest and Other most volatile.

Figure 3 shows Stage 1 reconstruction training, converging to a best validation MSE of 0.00920 over 50 epochs. Figure 4 shows qualitative reconstructions: the autoencoder captures tissue layout and staining patterns but loses fine cellular detail, consistent with the information bottleneck imposed by removing skip connections.

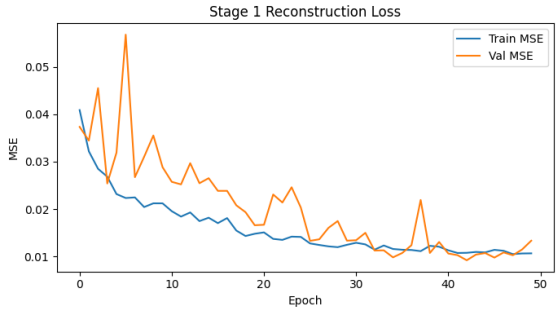


Figure 3: Stage 1 reconstruction loss.

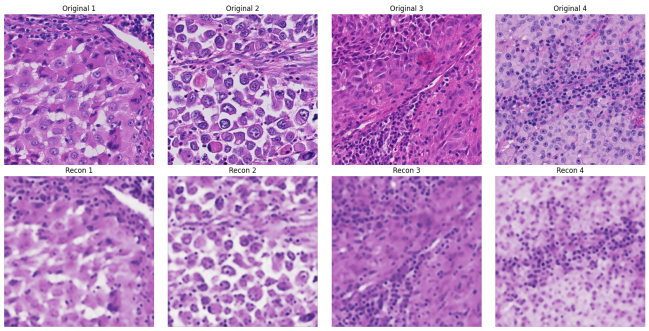


Figure 4: Stage 1 reconstructions (top: original, bottom: reconstructed). Tissue layout is preserved but fine detail is lost.

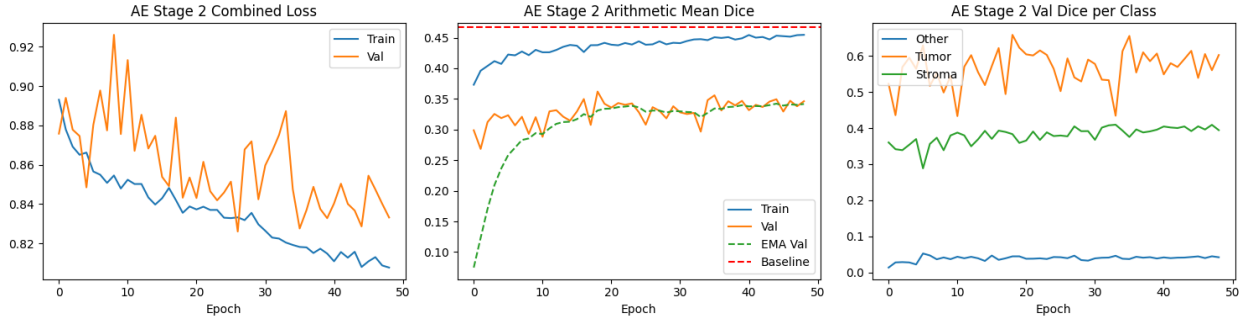
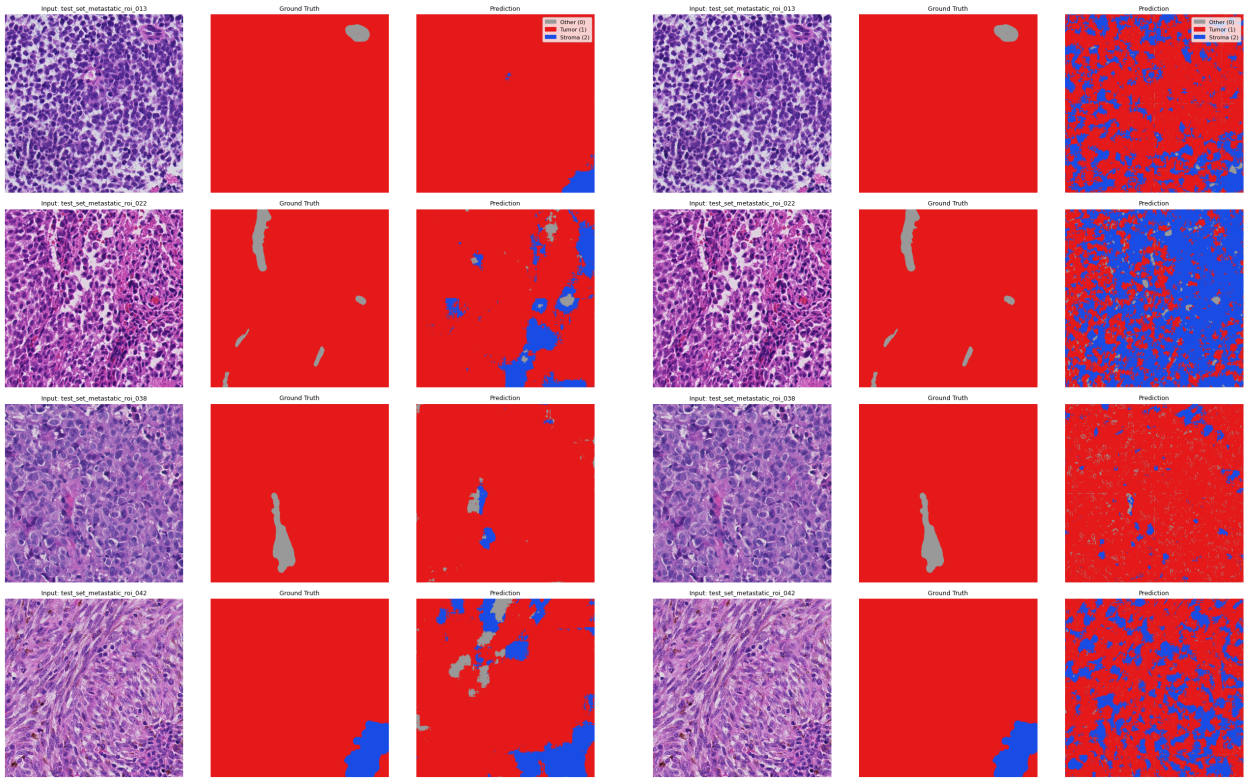


Figure 5: AE Stage 2 training curves. The validation Dice (centre) plateaus below the baseline (dashed red), with Other Dice (right) near zero throughout, indicating the frozen encoder's features are insufficient for discriminative segmentation.

Figure 5 shows Stage 2 segmentation training with the frozen encoder. The validation Dice plateaus around 0.35 and never reaches the baseline of 0.4670, with early stopping triggered at epoch 49. The train/val gap is smaller than the UNet's, which is expected since only the decoder is trainable, but the overall performance ceiling is substantially lower.

5.2 Qualitative Evaluation

Figure 6 compares both models on test images. The UNet produces spatially coherent predictions with well defined tumour/stroma boundaries and even correctly identifies some Other regions (grey patches in row 1). The autoencoder predictions are markedly more fragmented: Stroma (blue) is scattered across regions that should be Tumour, creating a noisy "salt and pepper" pattern rather than coherent tissue regions. This fragmentation is visible in all four test examples and reflects the frozen encoder's inability to produce features that discriminate tissue boundaries at the level needed for segmentation.



(a) UNet predictions (b) Autoencoder predictions

Figure 6: Test set qualitative comparison. Each column shows input, ground truth, and prediction.

5.3 Quantitative Evaluation

Table 3 reports test set performance. The UNet achieved a mean Dice of 0.6471, substantially above the 0.4670 baseline, while the autoencoder achieved 0.4170. Table 4 reports per class IoU.

Table 3: Test set Dice and pixel accuracy (global tiled inference, 22 images). Baseline mean Dice = 0.4670.

Model	Other	Tumour	Stroma	Mean Dice	Pixel Acc.
Baseline	n/a	n/a	n/a	0.4670	n/a
UNet	0.5007	0.8617	0.5790	**0.6471**	0.7780
AE + SegDecoder	0.0588	0.7737	0.4184	0.4170	0.6578

Table 4: Per class IoU on the test set.

Model	Other	Tumour	Stroma	Mean IoU
UNet	0.3339	0.7570	0.4074	0.4995
AE + SegDecoder	0.0303	0.6310	0.2646	0.3086

6 Discussion

6.1 Why the UNet Outperforms the Autoencoder

The UNet's substantial advantage (0.6471 vs 0.4170 mean Dice on test) can be attributed to three factors. First, **objective mismatch**: Stage 1 optimises MSE, rewarding the encoder for capturing pixel level colour and texture. Segmentation requires discriminative boundary features, which are structurally different. Raghu et al. [7] observed precisely this dynamic in medical imaging, where pre trained features transfer poorly when objectives are misaligned. The UNet avoids this entirely since every gradient update targets segmentation directly. Second, **frozen encoder limitation**: the encoder cannot adapt its reconstruction features toward boundary discrimination during Stage 2. The fragmented predictions in Figure 6(b) are a direct visual consequence of this. Third, **capacity**: the UNet trains 17.5M parameters end to end, while Stage 2 only trains 6.9M (the decoder), with no ability to refine the encoder's 10.6M frozen parameters.

6.2 Success and Failure Analysis

The UNet's strongest result is on Tumour (Dice 0.8617), which is expected given the 69.1% pixel dominance. More interesting is the UNet's 0.5007 on Other and 0.5790 on Stroma, demonstrating that the combination of class aware sampling, inverse frequency weighting, and Dice loss successfully elevated minority class performance. Figure 6(a) row 4 is a clear success case: the Stroma region in the lower right is correctly localised with a clean boundary, and small Other patches (grey) appear where expected.

The autoencoder's most striking failure mode is visible in Figure 6(b) rows 1 to 3: predictions resemble noise rather than coherent tissue regions, with Stroma scattered as small patches throughout Tumour areas. This is not random prediction but rather reflects the encoder's reconstruction features responding to local texture variation (cellular density, staining differences) that correlate weakly with tissue type, producing spatially inconsistent class assignments.

6.3 The Other Class Challenge

Other is semantically incoherent: it merges Blood Vessel (circular structures), Epidermis (layered cells), White Background (blank), and Necrosis (amorphous dead tissue) under one label. There is no shared visual pattern, since the class is defined by exclusion. Even with aggressive reweighting (CE weight 2.309), the autoencoder achieves only 0.0588 on Other. The UNet does substantially better (0.5007), suggesting that end to end training can learn the exclusionary decision boundary, whereas the reconstruction encoder cannot. This represents a fundamental label design constraint rather than a pure modelling failure.

6.4 Effect of Implementation Choices

Several implementation details proved critical. Computing class weights from raw masks rather than from the biased patch loader avoided double correction that would have over penalised Tumour. The EMA smoothing on the scheduler prevented premature learning rate decay caused by noisy validation signals from only 20 images. Keeping all inputs in $[0, 1]$ with no ImageNet normalisation ensured the encoder saw an identical input distribution in Stage 1 and Stage 2; without this, the encoder's learned features would be invalid when reused.

6.5 Limitations and Future Work

The primary limitation is dataset size: 163 training and 20 validation images produce noisy metrics and limit generalisation. The autoencoder's poor performance suggests that vanilla reconstruction pre training may be too weak a proxy for segmentation. Additionally, attention gate mechanisms [11] could improve minority class segmentation by directing decoder features toward spatially rare structures. Fine tuning the encoder in Stage 2 with a very low learning rate, rather than freezing it entirely, might also close the performance gap.

7 Task 2: Nuclei Classification

7.1 Problem Formulation

Task 2 aims to classify individual nucleus-centred image patches into one of three classes: tumour, lymphocyte, or histiocyte.

Each input sample is a 100×100 RGB patch centred on a single annotated nucleus. The objective is to learn a mapping $f : x \to y$, where $x \in \mathbb{R}^{100\times100\times3}$ and $y \in \{0, 1, 2\}$ corresponds to the three target classes.

This task is challenging because nuclei from different classes can appear very similar in shape, colour, and texture, despite representing different biological roles. The small patch size (100×100) further limits contextual information, so classification must rely on subtle local features. In addition, the original dataset is imbalanced, with histiocytes being significantly less frequent than other classes, which makes this class harder to model.

To address these challenges, we compare two approaches: a standard supervised classifier trained directly on labelled data, and a contrastive learning approach that first learns a structured representation before classification.

7.2 Dataset Construction

The dataset was prepared in three stages: patch extraction, creation of a balanced supervised set, and construction of a separate dataset for contrastive pretraining. All data is drawn from the PUMA dataset [1], which provides whole-slide H&E stained images with nucleus-level annotations.
Annotations are provided in GeoJSON format, where each entry corresponds to a nucleus with its spatial location and class label. Using the centroid of each annotation, a 100×100 RGB patch is extracted from the whole-slide image and assigned the corresponding label. Zero-padding is applied near image boundaries to preserve a consistent input size, matching the format of the given test set. Examples are shown in Figure 7.

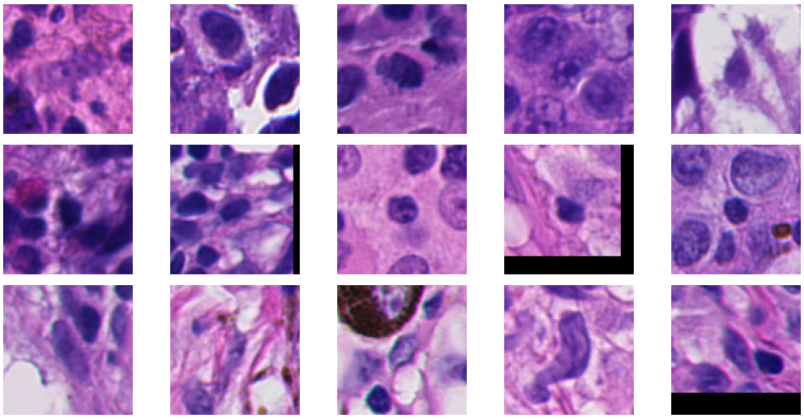


Figure 7: Example 100×100 nucleus patches from the dataset. Each row corresponds to a different class (*tumour*, *lymphocyte*, and *histiocyte*).

Next, a balanced dataset was created for supervised training. The original dataset is highly imbalanced, with histiocytes being much less frequent. To avoid bias towards majority classes, a fixed number of samples per class was selected: 2500 patches per class for training and 700 per class for validation, resulting in 7500 training and 2100 validation samples.
For contrastive pretraining, a separate dataset was constructed while avoiding data leakage. Samples used in supervised training were excluded, using the combination of slide ID and centroid coordinates to ensure uniqueness.
After filtering, the remaining nuclei were used to form a balanced contrastive dataset. Since histiocytes were the least frequent class, they determined the maximum number of samples per class, resulting in 3323 samples per class (9969 in total). We used all available samples because contrastive learning relies on pairwise comparisons within each batch. Using a larger dataset exposes the model to more diverse samples, which leads to a more reliable learning signal. However, this causes higher memory usage and longer training time.

7.3 Methodology

7.3.1 Supervised Classification Model

The supervised model learns a direct mapping from an input patch $\mathbf{x}$ to a predicted class label $\hat{y}$. Since the patches are small and contain only local information, the model must capture fine morphological and staining patterns while remaining robust to sample variation. For this reason, we use a ResNet18 architecture [**?**], which is expressive enough to learn useful features while remaining simple to train effectively on this task. Its residual connections also help stabilise optimisation.
The network takes a 100×100 RGB patch as input and uses the ResNet18 backbone as a feature encoder. The final feature representation is then passed to a task-specific classification head consisting of a dropout layer followed by a linear layer with three output units, corresponding to the classes *tumour*, *lymphocyte*, and *histiocyte*. The dropout layer is included to reduce overfitting and improve generalisation.
The model is trained from scratch rather than using pretrained weights. This is because pretrained models such as ImageNet are learned from natural images, which differ substantially from histopathology data in texture, colour distribution, and structure. This also ensures a fair comparison with the contrastive approach.
The model is trained using the cross-entropy loss, which applies a softmax over the predicted logits ($\hat{y}_c$) to obtain class probabilities.

$$\mathcal{L}_{\text{CE}} = -\log \frac{\exp(\hat{y}_y)}{\sum_{c=1}^{C} \exp(\hat{y}_c)},$$

Where $C = 3$ is the number of classes. The architecture is illustrated in Figure 8.

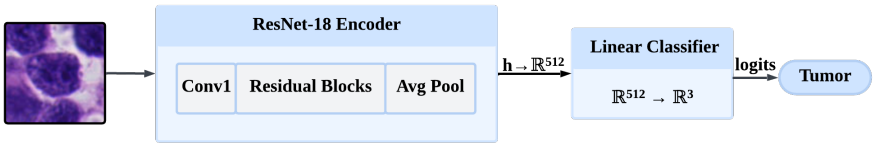


Figure 8: Supervised architecture for nuclei classification using a ResNet-18 encoder and a linear classifier.

7.3.2 Contrastive Learning Model

To better capture the underlying structure of the data, we use a supervised contrastive learning (SupCon) approach. Since class labels are available, a supervised contrastive formulation is preferred over an unsupervised one. This allows the model to form more meaningful and class-consistent clusters in the embedding space, instead of relying only on augmentations to define similarity, which may not always match the semantic differences between nucleus types.
The same ResNet18 architecture is used as the encoder to ensure a fair comparison with the supervised model. Given an input $\mathbf{x}$, two stochastic augmentations are applied to generate two different views of the same sample. Each view is passed through the shared encoder to produce a feature representation $\mathbf{h} = f(\mathbf{x})$, which is then passed through a projection head to obtain an embedding $\mathbf{z} = g(\mathbf{h})$. The projection head is implemented as a small MLP and is only used during the pretraining stage.
Training is performed in two stages. First, the encoder and projection head are trained jointly using the supervised contrastive loss, where samples from the same class are treated as positive pairs and samples from different classes as negative pairs. This encourages a structured embedding space in which same-class samples are closer together.
The supervised contrastive loss is defined as

$$\mathcal{L}_{\text{SupCon}} = \sum_i \frac{-1}{|P(i)|} \sum_{p \in P(i)} \log \frac{\exp(\mathbf{z}_i \cdot \mathbf{z}_p / \tau)}{\sum_{a \neq i} \exp(\mathbf{z}_i \cdot \mathbf{z}_a / \tau)},$$

Where $P(i)$ denotes the set of samples in the batch that share the same label as i, and τ is a temperature parameter controlling the sharpness of the similarity distribution.
In the second stage, the projection head is removed and the encoder is frozen. Rather than using only the final pretraining checkpoint, multiple checkpoints are evaluated with a linear probe on the validation set, and the best-performing encoder is selected. A new classification head, with the same architecture as the supervised model, is then trained on top of the frozen encoder using the supervised dataset. The overall pipeline is illustrated in Figure 9.

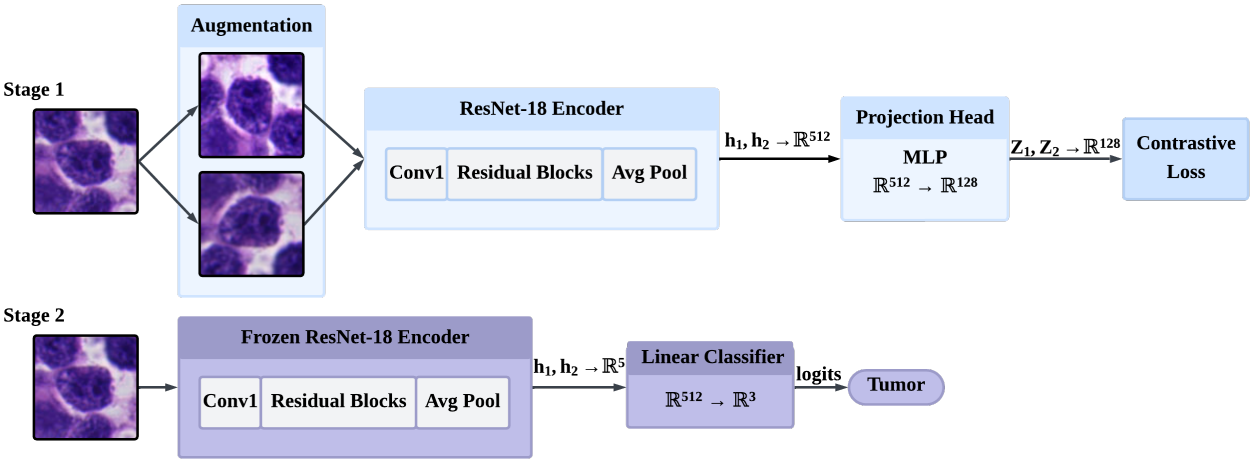


Figure 9: Model architecture for nuclei classification using supervised contrastive learning with augmented views and a ResNet-18 encoder.

7.4 Training and Evaluation Setup

All models are trained using AdamW with weight decay 10^{-4} and a cosine annealing learning rate schedule. Model selection is based on validation accuracy, and the best checkpoint is retained.
The supervised model is trained end-to-end with a learning rate of 10^{-3} and batch size 64 for up to 60 epochs with early stopping. To improve generalisation, label smoothing (0.05), dropout (0.2), and data augmentations, including random rotations, horizontal/vertical flips, and mild colour jitter, are applied to simulate staining and morphological variability.
The contrastive model is trained in two stages. In the pretraining stage, the encoder and projection head are trained using the supervised contrastive loss with learning rate 10^{-3} and batch size 128. Since the contrastive loss requires comparing each image against many others within the same batch, a larger batch size is used to provide more positive and negative pairs per update. Stronger augmentations are used to encourage invariance, and training is performed for 80 epochs.

In the second stage, the best encoder is selected using a linear probe, and a classification head identical to the supervised model is trained on top of the frozen encoder using the supervised dataset.
Performance is evaluated using accuracy and macro F1-score, where macro F1 is used to account for class imbalance by giving equal importance to each class. Class-wise precision, recall, and F1-score are also reported. To evaluate representation quality, k-nearest neighbour (kNN) accuracy, silhouette score, and t-SNE visualisations are used.

7.5 Results and Discussion

This section provides the quantitative and qualitative evaluation of the two nuclei classification approaches. Table 5 shows that both models perform similarly on the validation set, with the SupCon model having slightly higher accuracy and macro F1-score. The difference is more pronounced on the official test set, where the SupCon model outperforms the supervised model (0.7594 vs. 0.7180) and exceeds the baseline of 0.7083.
The SupCon model also performs slightly better on the test set than on validation. Since the datasets are strictly separated, this is more likely due to differences in data distribution and class imbalance, where the learned representation generalises better to unseen samples. In contrast, the supervised model shows a drop from validation to test, suggesting mild overfitting despite regularisation and augmentation techniques used.

Model	Val Acc	Val Macro F1	Test Acc	Test Macro F1
Baseline	–	–	0.7083	–
Supervised ResNet18	0.7310	0.7278	0.7180	0.7038
SupCon + Frozen Head	**0.7338**	**0.7348**	**0.7594**	**0.7508**

Table 5: Overall performance comparison on the validation and official test sets.

7.5.1 Comparison with Baseline and Parameter Efficiency

The baseline model contains approximately 5M trainable parameters. The supervised ResNet18 model used in this work has $11{,}178{,}051$ trainable parameters, which is much larger than the baseline. In contrast, the SupCon frozen-head model has the same total number of parameters but only $1{,}539$ remains trainable during the final classification stage, as the encoder is frozen after pretraining.

Model	Trainable Parameters	Total Parameters
Baseline	~ 5M	–
Supervised ResNet18	11,178,051	11,178,051
SupCon + Frozen Head	1,539	11,178,051

Table 6: Parameter comparison for the evaluated models.

Tables 6 and 5 show that the supervised model achieves only a small improvement over the baseline despite having more trainable parameters. This suggests that simply increasing the number of parameters does not necessarily lead to better generalisation. In this case, the model is quite large compared to the amount of available data, which increases its capacity, so it tends to memorise features instead of learning more general patterns, indicating overfitting. In contrast, the SupCon-based model achieves the best overall performance while training only a very small classification head in the final stage, indicating that most of the useful learning happens during contrastive pretraining.

7.5.2 Class-wise Performance on the Official Test Set

Table 7 reports class-wise precision, recall, and F1-score on the official test set. Several patterns can be observed.

Model	Class	Precision	Recall	F1-score
Supervised	Tumour	0.7601	0.7557	0.7579
	Lymphocyte	0.7825	**0.7557**	**0.7689**
	Histiocyte	0.5679	0.6026	0.5847
SupCon Frozen	Tumour	**0.8214**	**0.8343**	**0.8278**
	Lymphocyte	**0.8627**	0.6914	0.7676
	Histiocyte	**0.5853**	**0.7489**	**0.6571**

Table 7: Class-wise official test performance for the two models.

Both models classify *tumours* well, but SupCon improves both precision and recall, indicating better separation from other classes. For *lymphocyte*, SupCon achieves higher precision but lower recall, suggesting a more conservative

decision boundary with fewer false positives but more missed cases. *Histiocyte* remains the hardest class, yet SupCon improves recall from 0.6026 to 0.7489, giving a higher overall F1-score. Qualitative examples in Figure 10 support these trends by showing frequent confusion of lymphocyte and histiocyte with *tumour* under ambiguous morphology. First, both models are good at identifying *tumour* nuclei, but the SupCon improves both precision and recall, indicating better separation from other classes. Second, for *lymphocytes,* SupCon achieves higher precision but lower recall. This means it is more conservative for this class, predicting lymphocyte only when confident, which reduces false positives but increases missed cases. As a result of this trade-off, the overall F1-score remains very similar between the two models. Third, *histiocyte* is the hardest class for both models, which is consistent with its lower frequency in the original dataset. However, SupCon improves recall from 0.6026 to 0.7489, giving a higher overall F1-score. This behaviour is also supported by qualitative examples (Figure 10), where both models frequently misclassify lymphocyte and histiocyte as *tumour*, reflecting the bias towards the dominant class. Also, several ambiguous cases show disagreement between the two models, with the SupCon model correctly fixing some errors made by the supervised model.

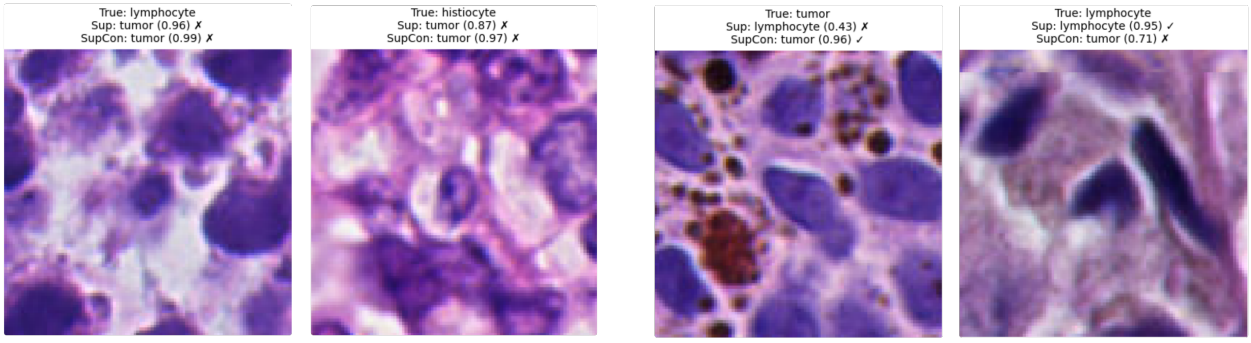


Figure 10: Qualitative examples from the test set illustrating representative failure and disagreement cases under ambiguous morphology.

7.5.3 Latent Space Quality

To evaluate representation quality, we analyse the learned features using silhouette score, k-nearest neighbour (kNN) accuracy, and t-SNE visualisations.

Model	Silhouette Score	kNN Accuracy
Supervised ResNet18	**0.1567**	0.7285
SupCon + Frozen Head	0.1393	**0.7823**

Table 8: Latent-space quality metrics extracted from test-set features.

Table 8 shows that the supervised model achieves a slightly higher silhouette score, while the SupCon model has a much higher kNN accuracy. This indicates that the supervised features create more clearly separated clusters overall, whereas the SupCon features group same-class samples more effectively, making the representation more discriminative. This is consistent with SupCon model's better classification performance and is also supported by the t-SNE visualisations in Figure 11.

7.5.4 Effect of Sample Type: Primary vs Metastatic

We further analyse performance by sample type using the primary/metastatic labels extracted from the test-set metadata during preprocessing. Primary samples come from the original tumour site, whereas metastatic samples come from tumours that have spread.

Model	Subtype	Accuracy	Macro F1
Supervised	Primary	0.7285	0.7141
	Metastatic	0.7014	0.6055
SupCon	Primary	**0.7504**	**0.7467**
	Metastatic	**0.7736**	**0.7002**

Table 9: Overall performance by sample type.

Table 9 shows that sample type affects performance. For the supervised model, primary samples are easier than metastatic ones, with both accuracy and macro F1 dropping on metastatic data. The larger drop in macro F1 means

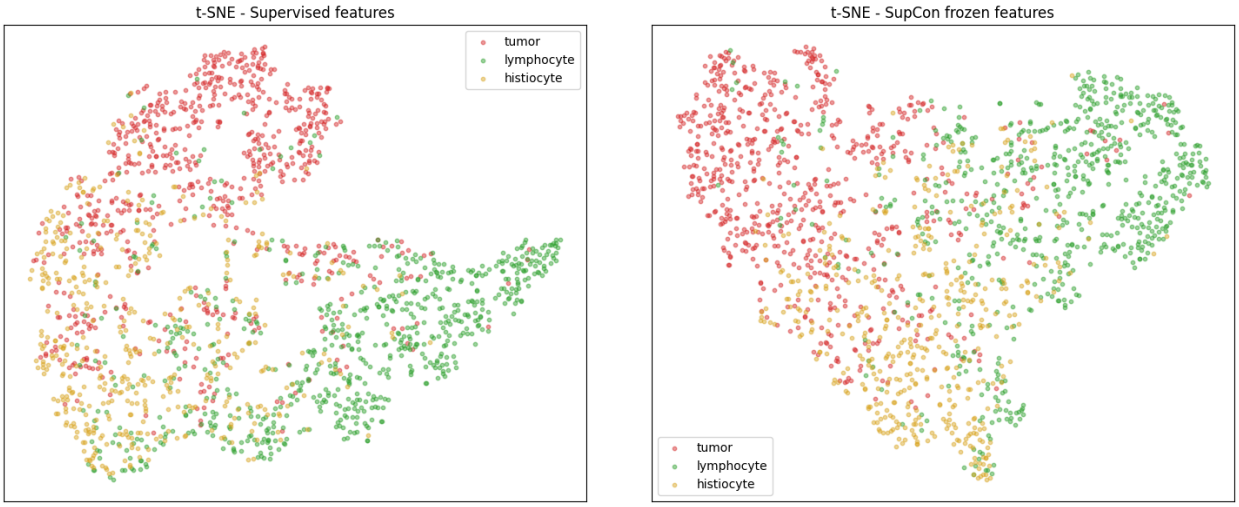


Figure 11: t-SNE visualisation of the learned feature space for the supervised model (left) and the SupCon-based model (right).

performance becomes less balanced across classes. This is also shown in Table 10, where metastatic histiocyte is the hardest, with an F1-score of only 0.3379, which lower the macro average.
In contrast, the SupCon model performs well on both sample types and even achieves slightly higher accuracy on metastatic samples. However, its macro F1 is still lower on metastatic data, showing that class-wise performance is less balanced. As shown in Table 10, the SupCon model improves metastatic histiocyte performance to 0.5387, which helps explain its better overall generalisation.

Model	Subtype	tumour F1	Lymphocyte F1	Histiocyte F1
Supervised	Primary	0.6849	**0.7981**	0.6593
	Metastatic	0.8000	0.6786	0.3379
SupCon	Primary	**0.7481**	0.7934	**0.6986**
	Metastatic	**0.8772**	**0.6846**	**0.5387**

Table 10: Class-wise F1-score by sample type.

7.5.5 Summary

According to the results, both methods outperform the baseline, but the SupCon model achieves the best overall performance, with higher test accuracy, macro F1-score, and better generalisation to metastatic samples. This improvement suggests that representation learning is particularly beneficial in settings with limited labelled data and subtle inter-class differences. By learning a structured embedding space before classification, the model reduces reliance on the final decision boundary and improves robustness to variations in appearance.

8 Statement of contribution

Student 1 did task 1 - coding and writing
Student 2 did task 2 - coding and writing
All models for task 1 were implemented in PyTorch [12], with geospatial mask rasterisation via `rasterio` [13], geometry processing via `Shapely` [14], and image loading via `tifffile` [15]. All models for task 2 were implemented in PyTorch [12] and torchvision [16], with patch extraction via Pillow [17] and tifffile [15], evaluation metrics via scikit-learn [18], and progress tracking via tqdm [19].

References

[1] Eleonora D'Arnese, Francesco Ciompi, Jakob Kather, and Andreas Kleppe. PUMA: Pan-cancer nuclei segmentation and classification. Grand Challenge Dataset, 2024. Available: `https://puma.grand-challenge.org`.

[2] Peter Bankhead, Maurice B. Loughrey, Jose A. Fernandez, Yvonne Dombrowski, Darragh G. McArt, Philip D. Dunne, Stephen McQuaid, Richard T. Gray, Liam J. Murray, Helen G. Coleman, Jacqueline A. James, Manuel Salto-Tellez, and Peter W. Hamilton. QuPath: Open source software for digital pathology image analysis. *Scientific Reports*, 7(1):16878, 2017.

[3] Alexander Buslaev, Vladimir I. Iglovikov, Eugene Khvedchenya, Alex Parinov, Mikhail Druzhinin, and Alexandr A. Kalinin. Albumentations: Fast and flexible image augmentations. *Information*, 11(2):125, 2020.

[4] David Tellez, Geert Litjens, Péter Bándi, Wouter Bulten, John-Melle Bokhorst, Francesco Ciompi, and Jeroen van der Laak. Quantifying the effects of data augmentation and stain color normalization in convolutional neural networks for computational pathology. *Medical Image Analysis*, 58:101544, 2019.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[6] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[7] Maithra Raghu, Chiyuan Zhang, Jon Kleinberg, and Samy Bengio. Transfusion: Understanding transfer learning for medical imaging. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.

[8] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(8):1798–1828, 2013.

[9] Carole H. Sudre, Wenqi Li, Tom Vercauteren, Sebastien Ourselin, and M. Jorge Cardoso. Generalised dice overlap as a deep learning loss function for highly unbalanced segmentations. In *Deep Learning in Medical Image Analysis Workshop, MICCAI*, 2017.

[10] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[11] Ozan Oktay, Jo Schlemper, Loic Le Folgoc, Matthew Lee, Mattias Heinrich, Kazunari Misawa, Kensaku Mori, Steven McDonagh, Nils Hammerla, Bernhard Kainz, Ben Glocker, and Daniel Rueckert. Attention U-Net: Learning where to look for the pancreas. In *Medical Imaging with Deep Learning (MIDL)*, 2018.

[12] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Köpf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch, 2024. `https://pytorch.org`.

[13] Sean Gillies et al. Rasterio: Geospatial raster I/O for Python programmers, 2024. `https://rasterio.readthedocs.io`.

[14] Sean Gillies et al. Shapely: Manipulation and analysis of geometric objects in the Cartesian plane, 2024. `https://shapely.readthedocs.io`.

[15] Christoph Gohlke. tifffile: Read and write TIFF files, 2024. `https://github.com/cgohlke/tifffile`.

[16] Sébastien Marcel and Yann Rodriguez. TorchVision: PyTorch's computer vision library, 2024. `https://pytorch.org/vision`.

[17] Alex Clark and contributors. Pillow: The friendly PIL fork, 2024. `https://python-pillow.org`.

[18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. scikit-learn: Machine learning in Python, 2024. `https://scikit-learn.org`.

[19] Casper da Costa-Luis et al. tqdm: A fast, extensible progress bar for Python, 2024. `https://tqdm.github.io`.